Deep Neural Networks for Lung Cancer Diagnosis

By: Miguel Magno and Grant Gonnerman

Lung Cancer

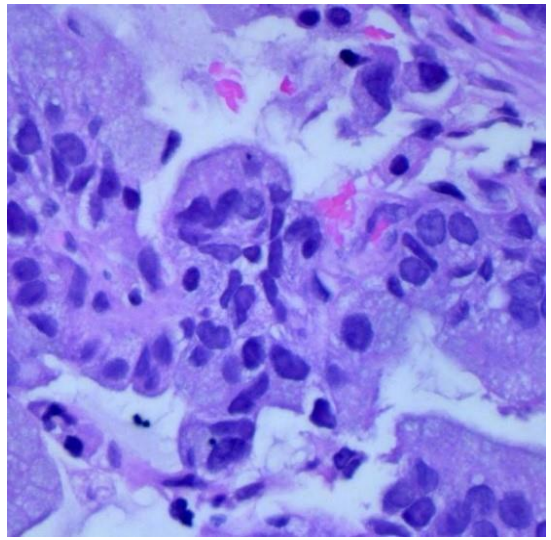
- Lung cancer is the third most common cancer in the United States.
- More Americans die from lung cancer than any other type of cancer across both men and women.
- When detected in an early stage, lung cancer is highly treatable but if it is misdiagnosed or not identified it may progress and therefore become harder to treat.
- It is common for early signs to be missed as a recent study found nearly 38% of all cancer-related medical malpractice claims were due to lung cancer misdiagnoses.
- We hope to create a convolutional neural network that can not only distinguish between healthy lungs and cancerous lungs, but also identify the kind of lung cancer present.

The Data

- This dataset was found on Kaggle but originally published by Andrew Borkowski M.D
- The dataset contains 15,000 images of three types of lungs. Each class contains 5,000 images.
- The three kinds are Adenocarcinoma, Squamous Cell Carcinoma, and Benign.
- The two kinds of lung cancer are forms of non-small cell lung cancer (NSCLC), a type of lung cancer that forms in the epithelial cells of the lungs
- NSCLC is the most common type of lung cancer, accounting for about 85% of all cases.

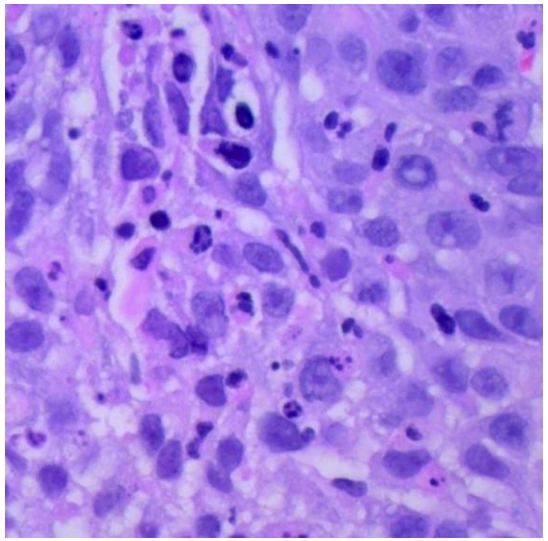
The Images

Adenocarcinoma



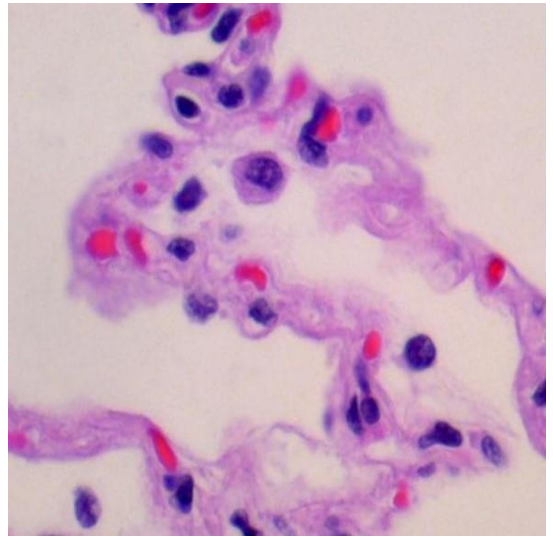
- The most common form of not only NSCLC, but all lung cancers

Squamous Cell Carcinoma



- The second most common form of lung cancer

Benign



- Images of healthy lungs
- Serve as a control group for comparison with the cancerous image

Baseline Model: Model 1

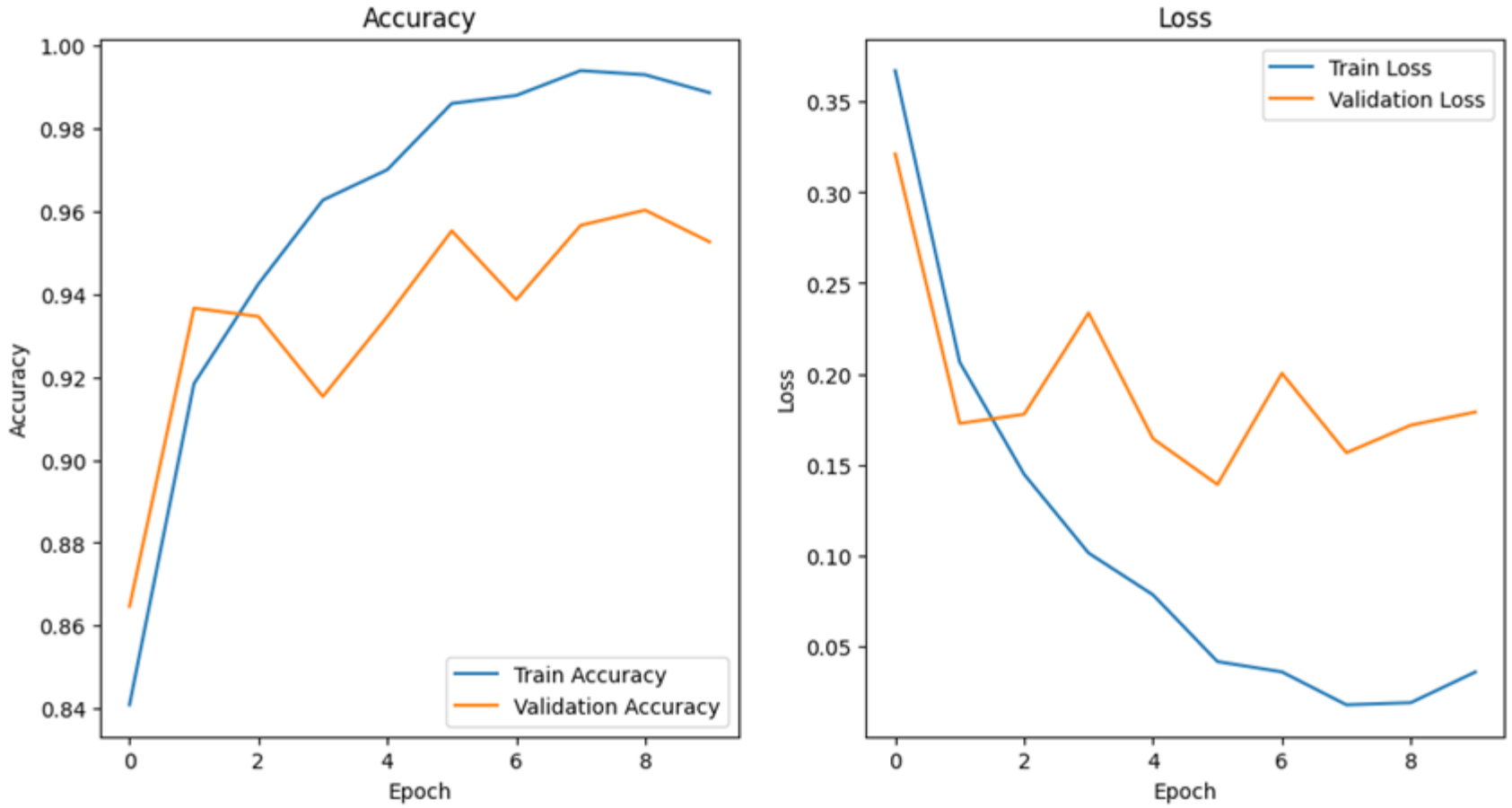
```
md1 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128,activation='relu'),
    tf.keras.layers.Dense(3,activation='softmax')
])
md1.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

history = md1.fit(train_ds, validation_data = val_ds, epochs=10)
```

Key Points:

1. Rescaling layer
2. 3 convolutional blocks
3. 1 dense layer
4. Main activation function is ReLU
5. Adam optimizer with default learning rate (0.001)
6. Validation accuracy of 95.2%

Baseline Training and Validation Results



Intermediate Model: Model 2

Key Points:

1. 4 convolutional blocks
2. Less neurons in the dense layer (128 -> 32)
3. Validation accuracy of 96.8%

```
md2 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(3,activation='softmax')
])
md2.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

history = md2.fit(train_ds, validation_data = val_ds, epochs=10)
```

Intermediate Model: Model 3

Key Points:

1. Dropout layer with rate of 0.2 added after the dense layer
2. Validation accuracy of 95.8%

```
md3 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(3,activation='softmax')
])
md3.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

history = md3.fit(train_ds, validation_data = val_ds, epochs=10)
```

Intermediate Model: Model 4

Key Points:

1. RMSprop optimizer used with the default learning rate (0.001)
2. Validation accuracy of 96.7%

```
md4 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(3,activation='softmax')
])
md4.compile(optimizer='RMSprop',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

history = md4.fit(train_ds, validation_data = val_ds, epochs=10)
```

Final Model

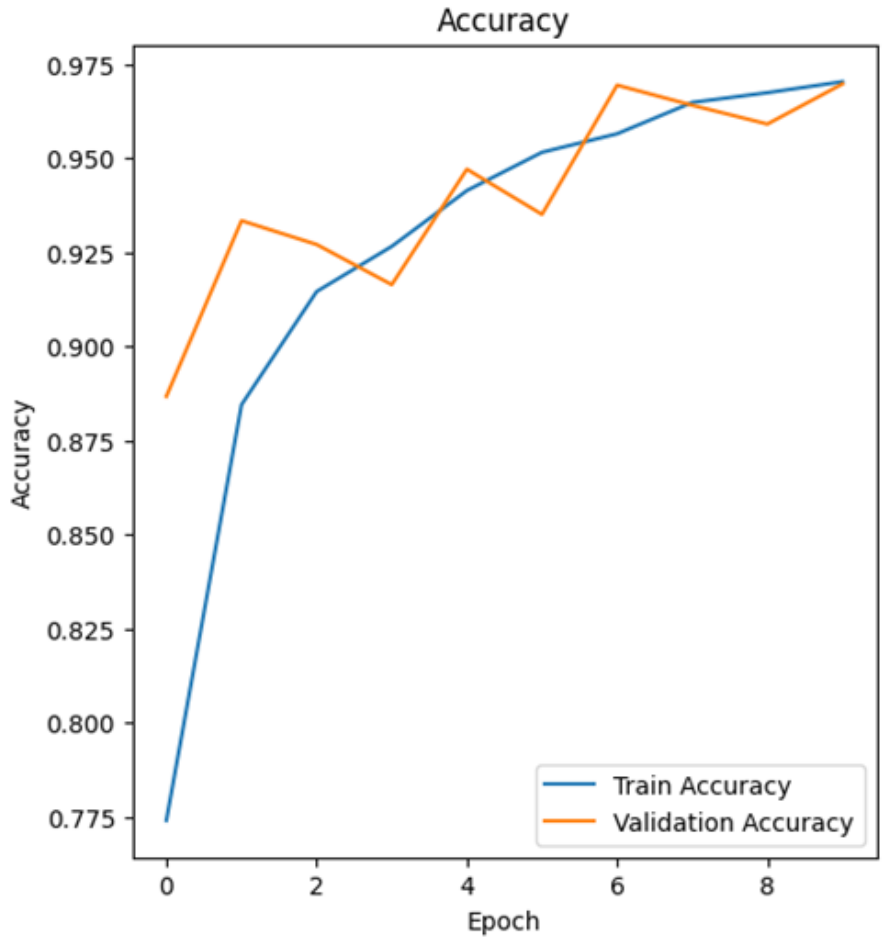
Key Points:

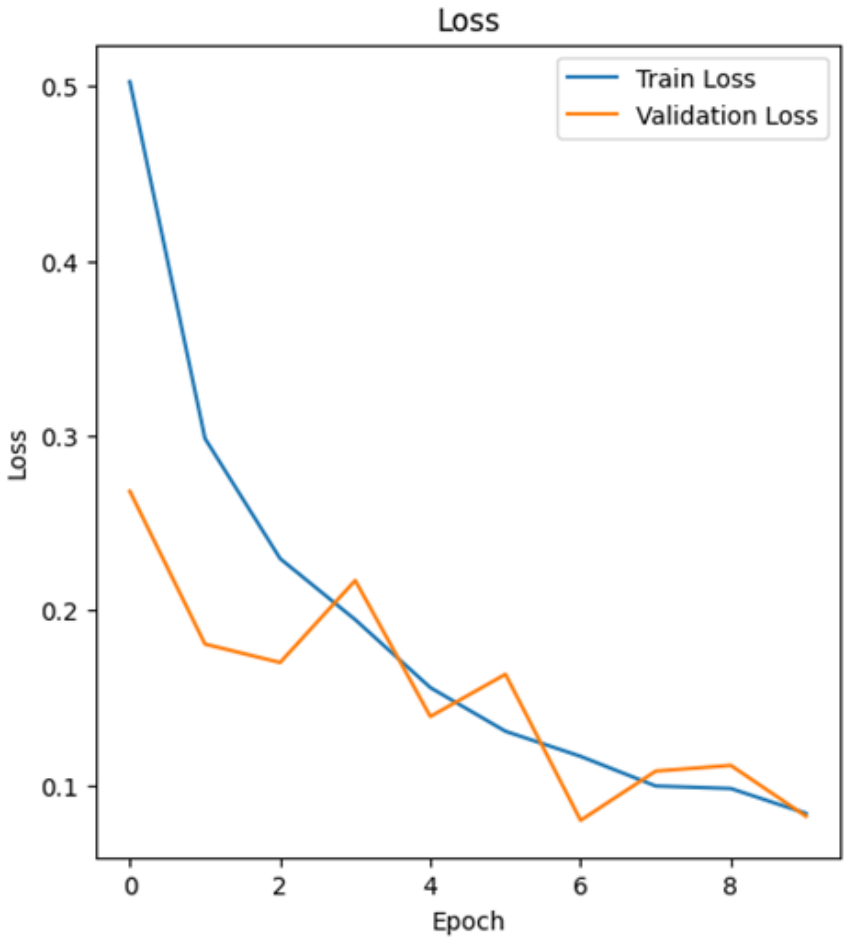
1. Compiled the best parts of each previous model
2. Validation accuracy of 96.9%

```
md5 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64,kernel_size=(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(3,activation='softmax')
])
md5.compile(optimizer='RMSprop',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

history = md5.fit(train_ds, validation_data = val_ds, epochs=10)
```

Final Training and Validation Results





Conclusion & Areas of Further Study

- Lung cancer CAN be reliably detected by neural networks based on our results with this dataset
- Our final model was able to reach a validation accuracy of 96.9%
- Achieved this by adjusting a wide variety of hyperparameters including optimizers, activation functions, image resolutions, dropout layers, and deeper networks
- Still more room for improvement however – we had a major limitation in the form of computational resource
  - More combinations of hyperparameters
  - Deeper networks
  - Higher number of epochs
  - Data augmentation
  - Multiple fold testing for every model

References

Data:

- https://www.kaggle.com/datasets/rm1000/lung-cancer-histopathological-images/data

Lung cancer background information:

- https://www.cdc.gov/lung-cancer/statistics/index.html
- https://www.verywellhealth.com/risks-of-misdiagnosed-lung-cancer-and-delayed-treatment-5248414
- https://www.cancer.org/cancer/types/lung-cancer/about/what-is.html